SalaryNet
Teaching Guide

Build, save, serve and deploy a simple neural network
with PyTorch, Flask and Google Cloud

A hands-on classroom walkthrough

1. The big picture

We will teach a computer to guess a person's salary from their years of experience. We do this with a **neural network** — a small mathematical function whose numbers (called **weights**) are adjusted automatically until its guesses match reality.

The whole journey has four stages:

- **Train** the network in a Jupyter notebook and **save** it to a file.
- **Serve** it with a Flask web app so anyone can get a prediction in a browser.
- **Package** everything with a requirements file and setup script.
- **Deploy** it to a Google Cloud Compute Engine virtual machine (a rented computer in a data centre).

> **Analogy:** training is like a student practising past papers. Saving the model is like writing down everything learned. Serving is opening a help desk where others can ask questions. Deploying is moving that help desk into a building the whole world can visit.

Files in the project

File	What it does
`model.py`	Defines the network. Shared by the notebook and the app.
`train.ipynb`	Trains the network and saves salary_net.pth.
`salary_net.pth`	The saved, trained weights.
`app.py`	Flask web app that loads the model and predicts.
`requirements.txt`	The exact Python packages needed.
`deploy/`	Script and service file to run it on a cloud VM.

2. What the network looks like

Our network, **SalaryNet**, is as small as a useful network gets. It takes **one number in** (years of experience) and gives **one number out** (salary in thousands of rupees).

- **Input layer:** 1 number — years of experience.
- **Hidden layer:** 2 neurons. Each makes its own weighted guess.
- **ReLU activation:** turns any negative value into 0. This small kink is what lets the network bend its line and learn the 'salary rises faster later' pattern.
- **Output layer:** 1 number — the predicted salary.

> A **neuron** just computes *(input × weight)* + *bias*. 'Learning' means nudging those weights and biases until the output is close to the real salaries.

model.py

```
import torch
from torch import nn

class SalaryNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.hidden = nn.Linear(1, 2)   # 1 input  -> 2 hidden
        self.relu   = nn.ReLU()         # activation
        self.output = nn.Linear(2, 1)   # 2 hidden -> 1 output

    def forward(self, x):
        h = self.hidden(x)   # weighted sums
        h = self.relu(h)     # activation
        return self.output(h)
```

The same file also has a `load_model()` helper that rebuilds the network and loads the saved weights — the app uses it.

3. Training the network (train.ipynb)

Open the notebook and run the cells top to bottom. Here is what each important cell does.

The data

```
X = torch.arange(0, 11).float().reshape(-1, 1)   # 0..10 years
y = torch.tensor([20,25,30,35,50,65,80,95,110,125,140]).float().reshape(-1,1)
```

X is the experience, y is the real salary in thousands. The network never sees the answer rule — it must discover it.

The training loop

```
loss_fn   = nn.MSELoss()                              # how wrong are we?
optimizer = torch.optim.Adam(model.parameters(), lr=0.02)

for epoch in range(3000):
    prediction = model(X)                 # 1. guess
    loss = loss_fn(prediction, y)         # 2. measure the error
    optimizer.zero_grad()                 # 3. reset old gradients
    loss.backward()                       # 4. how to improve?
    optimizer.step()                      # 5. nudge the weights
```

These five steps repeat 3000 times. Each pass is one **epoch**. The **loss** (the error) shrinks as the network improves.

- **MSELoss** — the average squared gap between guess and truth.
- **Adam** — the optimizer; it decides how big a nudge each weight gets.
- **lr=0.02** — the learning rate, the size of each nudge.

Saving the trained model

```
torch.save(model.state_dict(), MODEL_PATH)   # -> salary_net.pth
```

> We save only the **weights** (the `state_dict`), not the whole Python object. The app rebuilds the same SalaryNet and pours these learned weights back in. This is the normal, portable way to ship a model.

4. Serving predictions with Flask (app.py)

A trained model sitting in a file helps no one. **Flask** is a tiny web framework that turns our model into a website and an API.

The model is loaded **once** when the server starts — not on every request — so predictions are fast.

```
model = load_model()            # once, at startup

def predict_salary(experience):
    x = torch.tensor([[float(experience)]])
    with torch.no_grad():                    # no training now, just predict
        return model(x).item() * 1000        # thousands -> rupees
```

Endpoints (the different web addresses)

Address	Method	Purpose
`/`	`GET`	A web page with a form.
`/predict`	`POST`	Reads the form, shows the prediction.
`/api/predict`	`POST`	JSON in, JSON out — for other programs.
`/health`	`GET`	Returns 'ok'. Lets the cloud check it is alive.

Try it on your own computer

```
python app.py                # open http://localhost:8080

# or ask the API from a terminal:
curl -X POST http://localhost:8080/api/predict \
     -H 'Content-Type: application/json' \
     -d '{"experience": 5}'
# -> {"experience": 5, "salary": 65000}
```

5. Packaging for deployment

Before sending the app to a server we must list exactly what it needs so the server can rebuild the same environment.

The virtual environment (venv)

A **venv** is a private box of Python packages for this one project, so it never clashes with other projects on the machine.

```
python3 -m venv venv            # create the box
source venv/bin/activate        # step into it (Win: venv\Scripts\activate)
pip install -r requirements.txt # fill it from the list
```

requirements.txt

```
--extra-index-url https://download.pytorch.org/whl/cpu
torch==2.8.0      # the neural network library (CPU build = smaller)
flask==3.1.0      # the web framework
gunicorn==23.0.0  # a production-grade server to run Flask
```

> **Why gunicorn?** Flask's built-in server (`app.run`) is only for your laptop. In production we use **gunicorn**, which runs several copies ('workers') of the app and handles real traffic safely.

Pinning exact versions (`==2.8.0`) means the server installs the *same* packages you tested with — no surprises.

6. Deploying to a GCP Compute Engine VM

A **Compute Engine VM** is a computer you rent from Google that runs 24/7 in a data centre. We will put our app on it and open it to the internet.

Step 1 — Create the VM

- In the Google Cloud Console, go to **Compute Engine** → **VM instances** → **Create**.
- Pick a small machine such as **e2-small** and a **Debian** or **Ubuntu** image.
- Tick **Allow HTTP traffic**.

Step 2 — Open port 8080 in the firewall

Our app listens on port 8080, so we tell Google's firewall to allow it.

```
gcloud compute firewall-rules create allow-8080 \
    --allow tcp:8080 --target-tags salarynet
```

Step 3 — Copy the project up and run the setup script

```
gcloud compute scp --recurse . YOUR_VM:~/salarynet
gcloud compute ssh YOUR_VM

cd ~/salarynet
sudo bash deploy/startup.sh
```

The `startup.sh` script does all the server work for you: installs Python, copies the app to `/opt/salarynet`, builds the venv, installs the requirements, and registers the app as a background service.

Step 4 — The service keeps it running

We register the app with **systemd** (Linux's service manager) so it starts on boot and restarts if it crashes.

```
sudo systemctl status salarynet     # is it running?
sudo journalctl -u salarynet -f     # watch the logs live
```

> Now open **http://YOUR_VM_EXTERNAL_IP:8080** in a browser. Your students' neural network is live on the internet!

7. Recap & glossary

The journey, start to finish

- Define the network once in **model.py**.
- Train it in **train.ipynb**, saving **salary_net.pth**.
- Load and serve it with **app.py** (Flask).
- List dependencies in **requirements.txt**.
- Ship to a **GCP VM**, run by **gunicorn** under **systemd**.

Glossary

Term	Meaning
Neuron	A tiny formula: input × weight + bias.
Weight / bias	The numbers the network learns.
Epoch	One full pass over all the training data.
Loss	A number measuring how wrong the guesses are; we shrink it.
Optimizer (Adam)	The method that adjusts the weights to reduce loss.
state_dict	A dictionary of the model's learned weights that we save.
Flask	A small Python framework for building websites and APIs.
Endpoint	A web address the app responds to, e.g. /predict.
venv	A private, per-project box of Python packages.
gunicorn	A production web server that runs the Flask app.
systemd	Linux's service manager; keeps the app running and restarts it.
Compute Engine VM	A computer rented from Google Cloud, running all the time.